

Department of Computer Science and Engineering

Course Code &Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192472170
Student Name	G. Sirisha

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of “ShopSphere”, an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds $n = 100,000$ products and is expected to reach $n = 1,000,000$ within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice.

Every night, the following routine compares every product with every later product.

FOR $i \leftarrow 0$ TO $n - 2$ DO

FOR $j \leftarrow i + 1$ TO $n - 1$ DO

IF $Product[i].title = Product[j].title$ AND $Product[i].seller =$

$Product[j].seller$ THEN

MarkDuplicate(j)

END IF

END FOR

END FOR

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is $\Theta(n \log n)$ because the outer loop runs about $\log n$ times and the inner loop runs up to n times.

```

i ← 1

WHILE i <= n DO

    FOR j ← 0 TO i - 1 DO

        Audit(Request[j]) // basic operation

    END FOR

    i ← i × 2

END WHILE

```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies $T(n) = T(n/2) + 1$, $T(1) = 1$.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size $n/2$ (because of replicated partitions) and merges the partial results in linear time: $T(n) = 4T(n/2) + n$. A proposed redesign removes the replication, giving $T(n) = 2T(n/2) + n$.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving $T(n) = 2T(n/2) + n \log n$. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

Module	Input size	Dominant basic operation	Where it occurs	Growth
M1 – Duplicate Detector	$\sqrt{n}$ products	Comparing title and seller	Inner IF inside both loops	Quadratic

M2 – Rate Limiter	$\backslash(n\backslash)$ requests	Audit(Request[j])	Inner loop for each doubling of $\backslash(i\backslash)$	Linear
M3 – Product Search	$\backslash(n\backslash)$ products	Comparing search key with product key	Each search step	S1: linear, S2: logarithmic
M4 – Recommendation	$\backslash(n\backslash)$ customer-product records	Processing/merging partial results	At every recursive level	Current: quadratic; proposed: $\backslash(n\log n\backslash)$
M5 – Analytics	$\backslash(n\backslash)$ sales records	Sorting/processing $\backslash(n\log n\backslash)$ work at each level	Every recursive level	$\backslash(n\log^2 n\backslash)$

Explanation

- **M1:** Every product is compared with almost every other product, so the number of comparisons grows as $\backslash(n^2\backslash)$.
- **M2:** Although the outer loop doubles $\backslash(i\backslash)$, the total inner-loop work forms a geometric series.
- **M3:** S1 may inspect products one by one. S2 removes half the remaining search space after every comparison.
- **M4:** Four recursive calls create a large amount of repeated work in the current design.
- **M5:** The recursive splitting contributes $\backslash(\log n\backslash)$ levels, while each level performs $\backslash(n\log n\backslash)$ work.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's $\Theta(n \log n)$ claim is correct and give the tight bound. Verify your M2 result for $n = 1,024$.

M1 – Exact Number of Comparisons

The loops are:

$$i = 0 \text{ to } n - 2$$

For a fixed i , j runs from $i+1$ to $n-1$.

Therefore, the number of comparisons for each i is:

$n-i-1$

Hence,

$$C(n) = \sum_{i=0}^{n-2} (n-i-1)$$

Writing the terms:

$$C(n) = (n-1) + (n-2) + (n-3) + \dots + 1$$

Using the sum of first $n-1$ integers:

$$C(n) = \frac{n(n-1)}{2}$$

Time Complexity

$$\Theta(n^2)$$

Space Complexity

Only loop variables and temporary values are required.

$$\Theta(1)$$

If the duplicate marks/output are counted as extra storage, that output can require $O(n)$ space.

Interpretation

M1 becomes extremely expensive when the catalogue grows because doubling $\backslash(n\backslash)$ makes the work approximately four times larger.

M2 – API Rate Limiter

The values of i are:

$$1, 2, 4, 8, \dots, 2^k$$

where

$$2^k < n$$

For a particular i, the inner loop performs exactly i audits.

Therefore,

$$A(n) = 1 + 2 + 4 + \dots + 2^k$$

Using the geometric-series formula:

$$A(n) = 2^{k+1} - 1$$

where

$$k = \lfloor \log_2(n-1) \rfloor$$

Thus,

$$A(n) = 2^{\lfloor \log_2(n-1) \rfloor + 1} - 1$$

Therefore,

$$A(n) = \Theta(n)$$

Is the team's $\Theta(n \log n)$ claim correct?

No.

The inner loop does not run n times during every outer iteration. Its sizes are:

$$1, 2, 4, 8, \dots$$

Their sum is bounded by approximately $2n$.

Therefore:

$$M2 \text{ is } \Theta(n), \text{ not } \Theta(n \log n)$$

Verification for $n=1024$

Values of i :

$$1, 2, 4, \dots, 512$$

Therefore,

$$A(1024) = 1 + 2 + 4 + \dots + 512$$

$$= 2^{10} - 1$$

$$A(1024) = 1023$$

So only 1,023 Audit operations are performed.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

Given:

$$T(n) = T(n/2) + 1$$

and

$$T(1) = 1$$

Step 1

$$T(n) = T(n/2) + 1$$

Step 2

Substitute for $T(n/2)$:

$$T(n) = T(n/4) + 1 + 1$$

$$T(n) = T(n/4) + 2$$

Step 3

$$T(n) = T(n/8) + 3$$

Step 4

$$T(n) = T(n/16) + 4$$

After k substitutions:

$$T(n) = T\left(\frac{n}{2^k}\right) + k$$

We stop when:

$$\frac{n}{2^k} = 1$$

Therefore,

$$n = 2^k$$

Taking logarithm:

$$k = \log_2 n$$

Hence,

$$T(n) = T(1) + \log_2 n$$

Since $T(1)=1$:

$$T(n) = \log_2 n + 1$$

for $n=2^k$.

Time Complexity

$$\Theta(\log n)$$

Space Complexity

Iterative binary search:

$$\Theta(1)$$

Recursive binary search:

Each recursive call reduces n to $n/2$:

$$\Theta(\log n)$$

stack space.

Platform meaning

S2 is highly scalable because even a million-product catalogue requires only about 20 comparisons in the worst case.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a , b , $f(n)$, $n^{(\log_b a)}$ and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to $T(n) = 2T(n/2) + n \log n$; if not, derive the complexity using the extended Master Theorem or a recursion tree.

M4 – Current Design

Given:

$$T(n) = 4T(n/2) + n$$

Compare with:

$$T(n) = aT(n/b) + f(n)$$

Therefore:

$$a = 4, \quad b = 2, \quad f(n) = n$$

Calculate:

$$\begin{aligned} n^{\log_b a} &= n^{\log_2 4} \\ &= n^2 \end{aligned}$$

Now compare:

$$f(n)=n$$

with

$$n^{\log_b a} = n^2$$

Since:

$$n = O(n^{2-\epsilon})$$

for $\epsilon = 1$ this is **Master Theorem Case 1**.

Therefore:

$$\boxed{T(n) = \Theta(n^2)}$$

When data doubles

If $n \rightarrow 2n$:

$$\begin{aligned} T(2n) &\approx (2n)^2 \\ &= 4n^2 \end{aligned}$$

So the running time becomes approximately:

$$\boxed{4 \times}$$

M4 – Proposed Redesign

Given:

$$T(n) = 2T(n/2) + n$$

Therefore:

$$a = 2, \quad b = 2, \quad f(n) = n$$

Calculate:

$$n^{\log_b a} = n^{\log_2 2} = n$$

Thus:

$$f(n) = \Theta(n^{\log_b a})$$

This is **Master Theorem Case 2**.

Therefore:

$$T(n) = \Theta(n \log n)$$

When data doubles

Approximately:

$$\begin{aligned} T(2n) &\approx 2n \log(2n) \\ &= 2n(\log n + 1) \end{aligned}$$

So asymptotically the running time grows by about:

$$2 \times$$

rather than 4times.

Improvement

$$\Theta(n^2) \rightarrow \Theta(n \log n)$$

This is a major scalability improvement.

M5 – Extended Master Theorem

Given:

$$T(n) = 2T(n/2) + n \log n$$

Here:

$$a = 2, \quad b = 2$$

Therefore:

$$n^{\log_b a} = n$$

and

$$f(n) = n \log n$$

The three basic Master Theorem cases do **not directly apply**, because $f(n)$ contains an additional logarithmic factor.

Using the extended form:

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$

Here:

$$k=1$$

Therefore:

$$\begin{aligned} T(n) &= \Theta(n^{\log_b a} \log^{k+1} n) \\ &= \Theta(n \log^2 n) \end{aligned}$$

Thus:

$$\boxed{T(n) = \Theta(n \log^2 n)}$$

Recursion-tree verification

At level 0:

$$n \log n$$

At level 1:

$$2 \left(\frac{n}{2} \log \frac{n}{2} \right) = n \log \frac{n}{2}$$

At level 2:

$$n \log \frac{n}{4}$$

Therefore, at level i:

$$n \log \left(\frac{n}{2^i} \right)$$

Let:

$$L = \log_2 n$$

The total internal work is:

$$\begin{aligned} n[L + (L - 1) + (L - 2) + \cdots + 1] \\ = n \frac{L(L + 1)}{2} \end{aligned}$$

Therefore:

$$T(n) = \Theta(nL^2)$$

Hence:

$$T(n) = \Theta(n \log^2 n)$$

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p , and evaluate it for $n = 1,000,000$ and $p = 0.5$.

S1 – Sequential Search

Best Case

The product is the first item.

1 comparison

Worst Case

The product is the last item, or it does not exist.

n comparisons

Average Case

If the product exists, assume it is equally likely to occur at any position.

The expected comparisons are:

$$\begin{aligned} & \frac{1 + 2 + 3 + \dots + n}{n} \\ &= \frac{\frac{n(n+1)}{2}}{n} \\ &= \frac{n+1}{2} \end{aligned}$$

Including Probability p

Probability that product exists:

p

Probability that it does not exist:

$$1-p$$

Expected comparisons:

$$E[C] = p \left(\frac{n+1}{2} \right) + (1-p)n$$

Simplifying:

$$E[C] = \frac{pn+p}{2} + n - pn$$

Therefore:

$$E[C] = n - \frac{p(n-1)}{2}$$

For n=1,000,000, p=0.5

$$E[C] = 1,000,000 - \frac{0.5(999,999)}{2}$$

$$= 1,000,000 - 249,999.75$$

$$E[C] = 750,000.25$$

So S1 requires approximately:

$$750,000 \text{ comparisons on average}$$

S2 – Binary Search

Case	Condition	Comparisons
Best	Product is at middle	1
Average	Product is found after several halvings	$\Theta(\log n)$
Worst	Product is at deepest level or absent	$\lfloor \log_2 n \rfloor + 1$

For n=1,000,000:

$$\lfloor \log_2(1,000,000) \rfloor + 1$$

$$= 19 + 1$$

20 comparisons

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0 , and show that $f(n)$ is not $O(n)$.

We need to prove:

$$f(n) = \Theta(n^2)$$

According to the formal definition, there must exist positive constants c_1, c_2, n_0 such that:

$$0 \leq c_1 n^2 \leq f(n) \leq c_2 n^2$$

for all $n \geq n_0$.

Lower Bound

Since all terms are non-negative:

$$f(n) = 3n^2 + 10n \log_2 n + 50$$

Therefore:

$$f(n) \geq 3n^2$$

Choose:

$$c_1=3$$

Upper Bound

For $n \geq 2$:

$$\log_2 n \leq n$$

Therefore:

$$10n \log_2 n \leq 10n^2$$

Also, for $n \geq 2$:

$$50 \leq 12.5n^2$$

Hence:

$$\begin{aligned} f(n) &\leq 3n^2 + 10n^2 + 12.5n^2 \\ &= 25.5n^2 \end{aligned}$$

Choose:

$$c_2=26$$

and:

$$n_0=2$$

Thus:

$$3n^2 \leq f(n) \leq 26n^2$$

for $n \geq 2$.

Therefore:

$$\boxed{f(n) = \Theta(n^2)}$$

Prove $f(n)$ is not $O(n)$

Suppose:

$$f(n) = O(n)$$

Then there must be constants $c > 0$ and n_0 such that:

$$f(n) \leq cn$$

But:

$$f(n) \geq 3n^2$$

Therefore:

$$3n^2 \leq cn$$

For $n > 0$:

$$3n \leq c$$

This cannot remain true as n grows without bound.

Hence:

$$\boxed{f(n) \notin O(n)}$$

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for $n = 1,000,000$. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Server speed:

$$10^8 \text{ operations/sec}$$

Therefore:

$$\text{Time} = \frac{\text{Number of operations}}{10^8}$$

Nightly processing limit:

$$10 \text{ minutes} = 600 \text{ seconds}$$

M1

$$C(n) = \frac{n(n-1)}{2}$$

For:

$$n = 1,000,000$$

$$C(n) = 1,000,000(999,999)/2$$

499,999,500,000

operations.

Time:

$$\frac{499,999,500,000}{10^8}$$

Convert to minutes:

$$\sim 83.33 \text{ minutes}$$

✗ Fails the 10-minute requirement.

M2

For $n=1,000,000$:

$$k = \lfloor \log_2(999,999) \rfloor = 19$$

Therefore:

$$A(n) = 2^{20} - 1$$

$$\boxed{1,048,575 \text{ operations}}$$

Time:

$$0.01049 \text{ seconds}$$

✓ Easily meets the requirement.

S2

Worst-case comparisons:

$$\lceil \log_2(1,000,000) \rceil + 1$$

$$= 20$$

Therefore:

$$\sim 20 \text{ operations}$$

Time:

$$20/10^8$$

$$= 2 \times 10^{-7} \text{ seconds}$$

$$\sim 0.0000002 \text{ seconds}$$

✅ Extremely scalable.

M4 – Current Design

Using:

$$T(n) = 4T(n/2) + n$$

and $T(1)=1$, the recurrence gives approximately:

$$T(n) = 2n^2 - n$$

For $n=1,000,000$:

$$T(n) \approx 2(10^6)^2 - 10^6$$

$$\boxed{\approx 1,999,999,000,000}$$

operations.

Time:

$$1.999999 \times 10^2 / 10^8$$

$$\approx 19,999.99 \text{ seconds}$$

$$\boxed{\approx 333.33 \text{ minutes}}$$

❌ Fails badly.

M4 – Proposed Design

For:

$$T(n) = 2T(n/2) + n$$

with $T(1)=1$:

$$T(n) = n \log_2 n + n$$

For $n=1,000,000$:

$$\log_2(1,000,000) \approx 19.9316$$

Therefore:

$$T(n) \approx 1,000,000(19.9316) + 1,000,000$$

$$\approx 20,931,569 \text{ operations}$$

Time:

$$20,931,569/10^8$$

$$\approx 0.2093 \text{ seconds}$$

✔ Meets the requirement easily.

M5

From the recursion-tree analysis:

$$T(n) \approx n \left[\frac{\log_2 n (\log_2 n + 1)}{2} + 1 \right]$$

For:

$$n = 1,000,000$$

and:

$$\log_2 n \approx 19.9316$$

we get approximately:

$$T(n) \approx 209,599,497$$

operations.

Time:

$$\sim 2.10 \text{ seconds}$$

✓ Meets the requirement.

Final Scalability Table

Module	Complexity	Approx. operations at $n = 10^6$	Approx. time	10-min limit
M1	$\Theta(n^2)$	5.00×10^{11}	83.33 min	✗ Fail
M2	$\Theta(n)$	1.05×10^6	0.0105 s	✓ Pass
S2	$\Theta(\log n)$	~20	0.0000002 s	✓ Pass
M4 Current	$\Theta(n^2)$	2.00×10^{12}	333.33 min	✗ Fail
M4 Proposed	$\Theta(n \log n)$	2.09×10^7	0.209 s	✓ Pass
M5	$\Theta(n \log^2 n)$	$\approx 2.10 \times 10^8$	2.10 s	✓ Pass

Overall Decision

Redesign M1 and M4 immediately.

Keep M2 and S2 as they are.

Monitor M5 but no immediate redesign is required.

The key scalability issue is that quadratic algorithms become expensive very quickly. M1 and the current M4 design both exceed the 10-minute nightly limit, while the redesigned algorithms remain comfortably within the platform's processing budget.

Originality note: The solution above is written in original wording and structure rather than copied from a reference answer; however, an exact plagiarism percentage can only be determined by a plagiarism-detection system.